

# VoiceSign 个性化 ASR 后台

## 需求说明书 v2 (Go 独立重构)

做一套「越来越懂你」的语音输入理解后台 —— 独立服务、Go 实现、持续个性化、面向未来全双工

|      |                                   |      |                                        |
|------|-----------------------------------|------|----------------------------------------|
| 文档版本 | v2.0 (2026-10-03)                 | 文档状态 | 定稿待评审 (可再改)                            |
| 阅读对象 | 需求方 (本人)、开发实施人员                   | 配套材料 | 《VoiceSign Harness 全景审阅稿 v1》 (飞书文档)    |
| 技术栈  | Go 单二进制 / 独立服务 / JSON 配置 / 零第三方依赖 | 边界声明 | 本后台与 VoiceSign Harness 解耦，是独立的语音输入理解服务 |

### 本文档说明 (给开发人员)

这是一份可直接开工的需求说明书：它把 VoiceSign 项目全景审阅稿中全部与「个性化」相关的诉求，收敛为一个独立的 Go 后台服务 (VoiceSign 个性化 ASR 后台)。你只需要按本文档第 4 章功能清单与第 11 章验收标准实现即可；第 1-3 章讲清为什么这样做，第 9 章给出未来全双工演进路线，避免开发方向跑偏。

# 1. 项目定位与核心目标

## 1.1 一句话定义

VoiceSign 个性化 ASR 后台：一套用 Go 编写的独立后台服务，接收手机端传来的语音转写文本（及原始音频），完成「纠错 → 理解 → 个性化增强 → 输出标准化意图」，并且在使用过程中持续学习用户的口语、专名、习惯与偏好，越用越懂用户。

## 1.2 与 VoiceSign Harness 的关系（重要修正）

本模块是纯粹独立的后台，不并入 VoiceSign Harness 二进制，也不与其耦合。

- ASR 后台 = 语音输入理解层（把用户说的变成机器能执行的意图）；Harness = 任务执行层（把意图变成真实结果）。两层职责不同、迭代节奏不同，必须解耦。
- ASR 后台通过标准接口对外输出意图 JSON，Harness 只是它的一个消费方；未来任何语音入口（手机 App、桌面端、其他设备）都可以复用同一个 ASR 后台。
- 两套代码独立仓库、独立部署、独立数据目录；仅共享一套「意图 JSON 契约」，契约版本化、只增不改。

## 1.3 北极星目标（不可动摇）

- 越来越懂你：这是本后台存在的唯一理由。系统在使用中持续沉淀你的口语说法、专有名词、指代习惯、项目背景与偏好，个性化能力逐日增强，不依赖一次性配置。
- 语音能力很强：识别结果的纠错、标点、意图理解、控制语义（打断/暂停/撤销）、语音文字双通道都要做到可用、可靠，体验对标豆包语音体系。
- 响应极快：本地规则优先，模型兜底，处理链路微秒到毫秒级，语音说完即出结果，不出现长等待。
- 面向未来全双工：架构从第一天就为「全双工实时语音交互」留好接口与数据模型（见第 9 章），未来实现边说边理解、随时打断、实时反馈。

# 2. 核心设计思想：个性化的三层递进

「越来越懂你」不是一句口号，而是三层能力逐层递进、每一层都在沉淀数据、数据反过来增强下一层的闭环：

| 层级               | 要解决的问题                                                    | 沉淀的个性化数据                                                    |
|------------------|-----------------------------------------------------------|-------------------------------------------------------------|
| L1 听清<br>(文本纠错层) | 语音转写常有错字、同音字、口语碎片、专名错误。例：「季总」实际是「冀总」；「那个 module」实际是某个项目名。 | 自定义词典（口语→标准写法）、纠错经验库、语音置信度标记。                               |
| L2 听懂<br>(语义理解层) | 一句话的意图是什么？「它」指什么？「那个文档」是哪个？结合上下文与历史才能正确理解。                | 指代消解缓存（意图/域/权限/指代四元组）、对话上下文记忆、意图置信度。                        |
| L3 懂你<br>(个人画像层) | 同一个词在不同人嘴里含义不同；同一件事在不同人场景下期望不同。需要结合个人偏好、项目知识、决策历史做个性化管理。  | 用户偏好 prefs、项目地图 project-map、决策日志 decisions-log、领域知识、使用模式统计。 |

核心公式：个性化 = 数据 × 规则 × 反馈闭环

- 数据：每一次语音输入、每一次用户确认、每一次纠错都留痕（append-only），形成私有复利资产。
- 规则：本地规则（词典、缓存、模式）优先，零模型调用，保证速度与确定性。
- 反馈闭环：用户改一个字、确认一个指代、纠正一次识别 —— 系统立即学习并固化，下次直接用，越用越准。

### 3. 系统架构（独立服务）

#### 3.1 总体架构



#### 3.2 分层设计原则

- 单向依赖：管线 1→8 顺序固定，每一层只依赖前一层输出，不反向调用。
- 本地规则优先，模型兜底：只有本地规则无法解决时，才调用模型中心；模型调用带超时，超时即降级，绝不阻塞。
- 个性化数据与代码分离：全部个性化知识存 JSON/JSONL 文件，代码只提供处理逻辑，两者独立演进。
- 输出契约稳定：对外只输出标准意图 JSON，任何消费方（Harness、测试、未来应用）按契约接入。

### 4. 功能需求清单（开发实施范围）

#### 4.1 基础服务能力 P0（必须）

| 需求项   | 具体要求                                                                |
|-------|---------------------------------------------------------------------|
| 独立服务  | Go 编写、独立进程/二进制部署，与 Harness 无代码耦合；提供 HTTP 接口 + 未来 WebSocket。         |
| 原始留底  | 每次输入永久保存 input_raw（原始转写文本），append-only，不可篡改；后续所有纠错/清洗都是在其上的派生，便于归因。 |
| 全链路日志 | 每一步处理（留底→清洗→纠错→上下文→指代→意图→域建议）写入轨迹 JSONL，带耗时与命中来源，可完全复现。             |
| 性能基线  | 本地规则路径（无模型调用）目标 < 1ms；含模型兜底路径设超时（默认 3s）与并发保护，超时自动走降级。               |
| 配置    | 全部 JSON 文件，不用 YAML；支持热加载（词典/偏好变更即时生效）。                              |

## 4.2 个性化词典 P0 (L1 听清的核心)

- 条目结构: {raw\_speech: ■■/■■, target: ■■■■, scope: ■■|■■, priority: ■■■, source: ■■}。
- 典型场景: 人名 (「季总→冀总」)、项目名、专有名词、口头禅映射、方言口音修正。
- 支持语音指令动态增删改: 用户说「记住, 冀总是冀中的冀」「删掉这个词」, 后台解析为词典操作并落盘。
- 匹配策略: 全词优先、最长匹配优先; 同词多条命中取最高优先级; 命中后标记纠错来源与置信度。
- 纠错安全: 高置信纠错直接替换; 低置信纠错输出「疑似纠错」标记, 交上层回问, 绝不静默改错。
- 词典可导出/导入, 版本可回溯 (append-only + 快照)。

## 4.3 文本清洗与纠错 P0

- 去除 ASR 常见的语气词、重复词、口误碎片 (「嗯」「那个」「就是就是」), 保留语义。
- 标点恢复: 为转写文本补全标点, 提升下游意图解析准确率 (M7 真机遗留项, 必须闭环)。
- 识别置信度标记: 对 ASR 引擎返回的低置信片段打标, 进入纠错或回问流程, 不直接执行。
- 清洗全过程保留原始输入, 输出带清洗说明 (改了哪里、为什么)。

## 4.4 指代消解 P1 (L2 听懂的核心)

| 层级          | 机制                                                  |
|-------------|-----------------------------------------------------|
| 第一层: 词典/实体库 | 「它=最近提到的实体」「这个项目=当前域项目」, 本地实体索引, 零模型调用, 100% 确定才替换。 |
| 第二层: 上下文规则  | 读取最近 N 轮对话上下文 (四元组缓存 {意图, 域, 权限, 指代}), 按规则匹配指代对象。   |
| 第三层: 模型兜底   | 仅前两层无法解决时调用模型推断; 输出带置信度。                            |

- 低置信度指代 → 输出 need\_disambiguate 标记, 触发回问确认 (一屏一决策, 只问一次)。
- 用户确认后自动固化进缓存, 下次直接复用; 缓存带过期策略 (如 7 天) 与容量上限。
- 「打断/暂停/撤销」三个控制语义必须识别并优先于普通指代处理。

## 4.5 上下文个性化记忆 P1 (L3 懂你的核心)

- 数据源: user-preferences.json (偏好)、project-map.json (项目地图)、decisions-log.jsonl (历史决策)、领域知识库。
- 用法: 根据当前输入做相关性检索, 只把相关片段注入解析过程, 绝不全量加载历史。
- 检索结果只做理解增强, 不直接改写原始输入; 输出 JSON 中携带命中的上下文来源。
- 异步低频更新 (偏好/项目知识变更后增量更新索引), 不阻塞实时请求。
- 使用模式统计: 高频说法、常见意图、活跃时段、确认率 —— 反哺词典权重与确认策略。

## 4.6 意图解析与标准化输出 P0

- 输出标准化意图 JSON: {type, path, params, confidence, need\_disambiguate?, domain\_suggestion?, control?}。
- 意图类型对齐 Harness 现有 8 类 + ORCHESTRATE (EDIT / DEBUG / QUERY / TEST / COMMIT / DEPLOY / NOTE / ASK / ORCHESTRATE), 兼容现有消费方。

- 控制语义独立输出：识别到「停/暂停/撤销」等，直接返回控制意图，不进入业务执行。
- 规则优先分类（关键词/词典/模式），模型兜底；置信度低于阈值 → 回问或转 ASK。

4.7 域 (Space) 建议 P1

- 根据意图与资源路径输出候选域建议（如 project / sandbox / vault-notes / global），交由上层权限门禁裁决。
- 原则：默认拒绝；未知意图落到 global 只读域，绝不猜测高权限域。
- ASR 后台只建议、不授权；权限校验永远在消费方（Harness 的 Space.Check）执行，本服务不绕过。

4.8 反馈学习闭环 P1（越用越懂你的机制）

| 反馈来源                      | 学习动作                              |
|---------------------------|-----------------------------------|
| 用户手动修改识别结果<br>(语音文字双通道)   | 差异分析 → 生成/强化词典条目（带来源与时间）→ 下次直接命中。 |
| 用户确认/纠正指代                 | 固化四元组缓存，提升对应指代规则权重。               |
| 用户确认/否认纠错建议               | 调整词典条目优先级；否认次数超阈值 → 条目降权或移除。      |
| 下游执行反馈<br>(Harness 报错/成功) | 写入异常知识库 / 使用模式统计，反哺词典与意图权重。       |
| 高频反复确认<br>(确认疲劳)          | 记录确认行为，缓存确认结果、限制重复弹窗；不可逆操作仍强制确认。  |

所有学习动作可审计：来源、时间、影响条目、是否生效，全部留痕。

4.9 异常与降级 P2（中期）

- 内部错误（词典文件损坏、缓存异常、解析失败）→ 错误指纹 → 对接诊断链路（可回写异常知识库）。
- fail-open：任何个性化组件故障时自动跳过，返回基础处理结果，保证语音输入链路 100% 有响应。
- 模型兜底调用失败/超时 → 降级为本地规则结果 + 标记 degraded，不阻塞、不空回。

5. 数据模型与持久化

全部本地 JSON/JSONL 存储，append-only，重启不丢；独立数据目录（如 ~/.voicesign/asr/），与 Harness 数据分离。

| 文件                         | 内容                        | 类型                   |
|----------------------------|---------------------------|----------------------|
| custom-dictionary.json     | 个性化词典条目（口语→标准，作用域/优先级/来源） | JSON + 快照            |
| context-memory.json        | 指代四元组缓存、对话上下文、确认固化结果      | JSON，带过期             |
| user-preferences.json      | 用户偏好（确认策略、默认域、纠错强度等）      | JSON                 |
| project-knowledge.json     | 项目地图、领域术语、决策摘要（供相关性检索）    | JSON                 |
| exceptions-knowledge.jsonl | 异常知识库（错误指纹→根因→修正经验）       | JSONL<br>append-only |

| 文件                  | 内容                      | 类型                   |
|---------------------|-------------------------|----------------------|
| traces-asr.jsonl    | 全链路轨迹（输入/每步处理/耗时/来源/输出） | JSONL<br>append-only |
| usage-patterns.json | 使用模式统计（高频说法/意图/时段/确认率）  | JSON 聚合              |

## 6. 对外接口定义（示意）

```
POST /v1/process      # 请求 → 处理 → JSON 响应
POST /v1/correct      # 修正请求 → 修正结果 + 置信度
POST /v1/dictionary   # 词典更新请求
POST /v1/feedback     # 反馈请求/来源/置信度 → 处理结果
GET /v1/health        # 健康检查
WS /v1/stream         # 流式数据 9 帧
```

请求示例 (/v1/process) :

```
{ "text": "把那个模块的错误提示改成中文", "session_id": "s1",
  "audio_meta": { "engine": "system", "confidence": 0.86, "lang": "zh" } }
```

响应示例:

```
{ "type": "EDIT", "path": "modules/quote",
  "params": { "action": "i18n", "target": "zh" },
  "confidence": 0.88, "domain_suggestion": [ "project:voicesign-harness" ],
  "traces": [ { "step": "clean", "ms": 0.2 },
               { "step": "dict", "hit": "模块→报价模块", "ms": 0.3 } ] }
```

## 7. 非功能需求

| 维度  | 要求                                                  |
|-----|-----------------------------------------------------|
| 性能  | 本地路径 < 1ms；模型兜底超时 3s 且并发受限；绝不阻塞语音输入链路。              |
| 可靠性 | fail-open 兜底；数据 append-only 不丢历史；重启快速恢复。            |
| 可观测 | 轨迹 JSONL 全链路留痕；每步耗时、命中来源、置信度可查。                     |
| 隐私  | 个性化数据全部本地存储，不上传；用户可导出/清除个人数据。                       |
| 安全  | 接口鉴权（本地回环优先）；不执行任何业务动作，天然低风险面。                      |
| 兼容  | 意图 JSON 契约与 Harness 现有 8 类 + ORCHESTRATE 完全兼容，只增不改。 |

## 8. 边界与红线（开发必须遵守）

- 只做语音输入理解，不做业务执行：本服务永不直接执行任务（不改文件、不提交、不部署），执行永远在消费方。
- 不并入 Harness：独立仓库、独立部署、独立数据目录，仅共享意图契约。
- 不做无人自治：高风险个性化修改（删除词典、改偏好、扩权限）仍需用户确认；低置信一律回问，不静默执行。
- 不无界膨胀：缓存/知识库必须有容量上限与过期策略，防止文件无限增长。

- 纠错不过度：纠错必须带置信度与来源，低置信不直接替换用户原意。
- 不依赖外部模型做关键路径：模型只做兜底，核心路径本地规则完成。

## 9. 未来能力演进路线图（全双工愿景）

架构从第一天为以下能力预留接口与数据模型，按里程碑逐步实现：

| 阶段         | 能力                              | 说明                                                  |
|------------|---------------------------------|-----------------------------------------------------|
| P0<br>立即可做 | 独立服务 + 词典纠错 + 意图 JSON + 语音文字双通道 | 先跑通「语音→意图」主线，纠错与双通道闭环（含标点恢复）。                       |
| P1<br>近期   | 指代消解 + 上下文注入 + 反馈学习闭环 + 域建议     | 三层理解全部上线，越用越懂你的复利机制开始运转。                            |
| P2<br>中期   | 流式处理（WS）+ 实时纠错预览 + 多语言（中/英/阿）   | 边说边出结果（partial results），体验向全双工过渡。                   |
| P3<br>远期   | 全双工实时语音交互（Full-Duplex）          | 用户说话过程中系统持续理解、可随时打断、实时反馈；支持多轮连续对话不重启；语音能力全面对标豆包并超出。 |

全双工（Full-Duplex）定义

用户与系统可以同时「说话」，系统在用户话语未说完时即开始理解，用户可随时打断系统，系统实时给出反馈——这是「语音能力很强」的终极形态，也是本后台长期演进方向。当前阶段先把文本级个性化理解做扎实，全双工基于同一套个性化数据与意图契约自然生长。

## 10. 风险与开发注意点

| 风险             | 对策                               |
|----------------|----------------------------------|
| 个性化数据无限膨胀      | 容量上限 + 过期策略 + 聚合统计，定期压缩归档。       |
| 词典纠错过度/改错原意    | 置信度分级：高置信替换、低置信回问；来源留痕可回滚。       |
| 模型兜底拖慢响应       | 本地优先 + 超时 + 并发限制 + fail-open 降级。 |
| 与 Harness 契约漂移 | 意图契约版本化、只增不改；CI 校验两侧 schema 一致。  |
| 旧数据迁移          | v1 阶段若有词典/缓存残留，提供迁移脚本，不直接丢弃。     |
| 成为第二个「执行引擎」    | 红线约束：永不执行业务动作；权限永远在消费方。          |

## 11. 验收标准

- Go 独立服务可编译、可部署，与 Harness 无代码耦合；仅通过意图契约对接。
- 输入转写文本 → 输出标准意图 JSON，可直接被 Harness 消费；8 类 + ORCHESTRATE 全部兼容。
- 词典支持语音指令增删改，立即生效且持久化，重启不丢。
- 标点恢复、清洗、纠错全链路有留痕与明细，可复现。
- 指代消解三层机制运行；低置信输出 need\_disambiguate 并触发回问；确认后固化复用。
- 反馈学习闭环：手动改字/确认/否认 → 词典与缓存更新 → 下次命中。

- 模型兜底超时/失败时，系统不卡死，fail-open 返回基础结果并标记 **degraded**。
- 全链路轨迹 JSONL 记录每一步处理与耗时；本地路径性能 < 1ms（基准可测）。
- 数据目录独立、格式为 JSON/JSONL、append-only，可导出可清除。

本文档由 VoiceSign 项目全景审阅稿 v1 中的全部个性化诉求整理而成，并依需求方最新指示修正：ASR 后台为独立服务、核心目标为个性化语音识别（越用越懂你）、面向未来全双工。文档可继续修订。